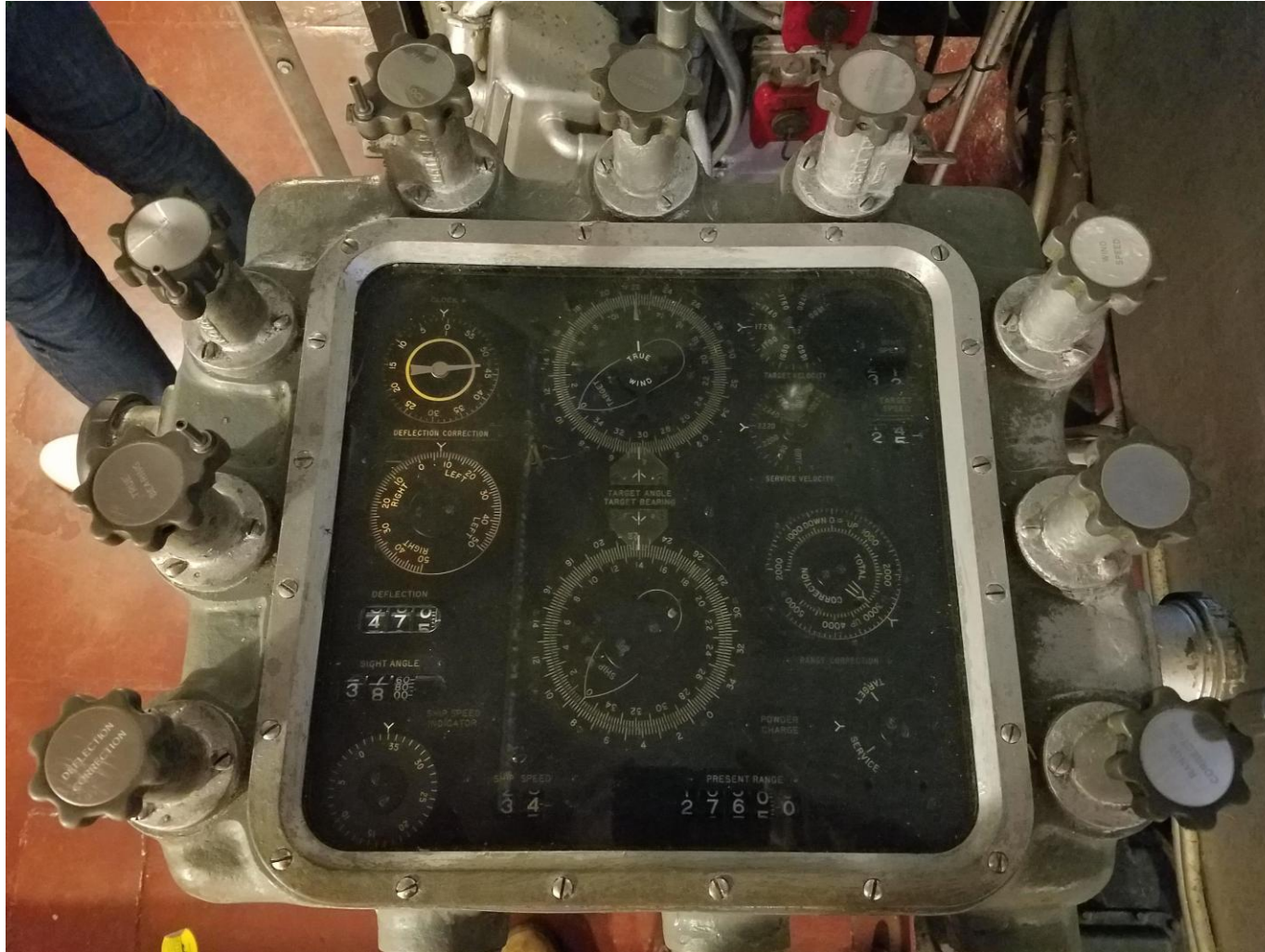


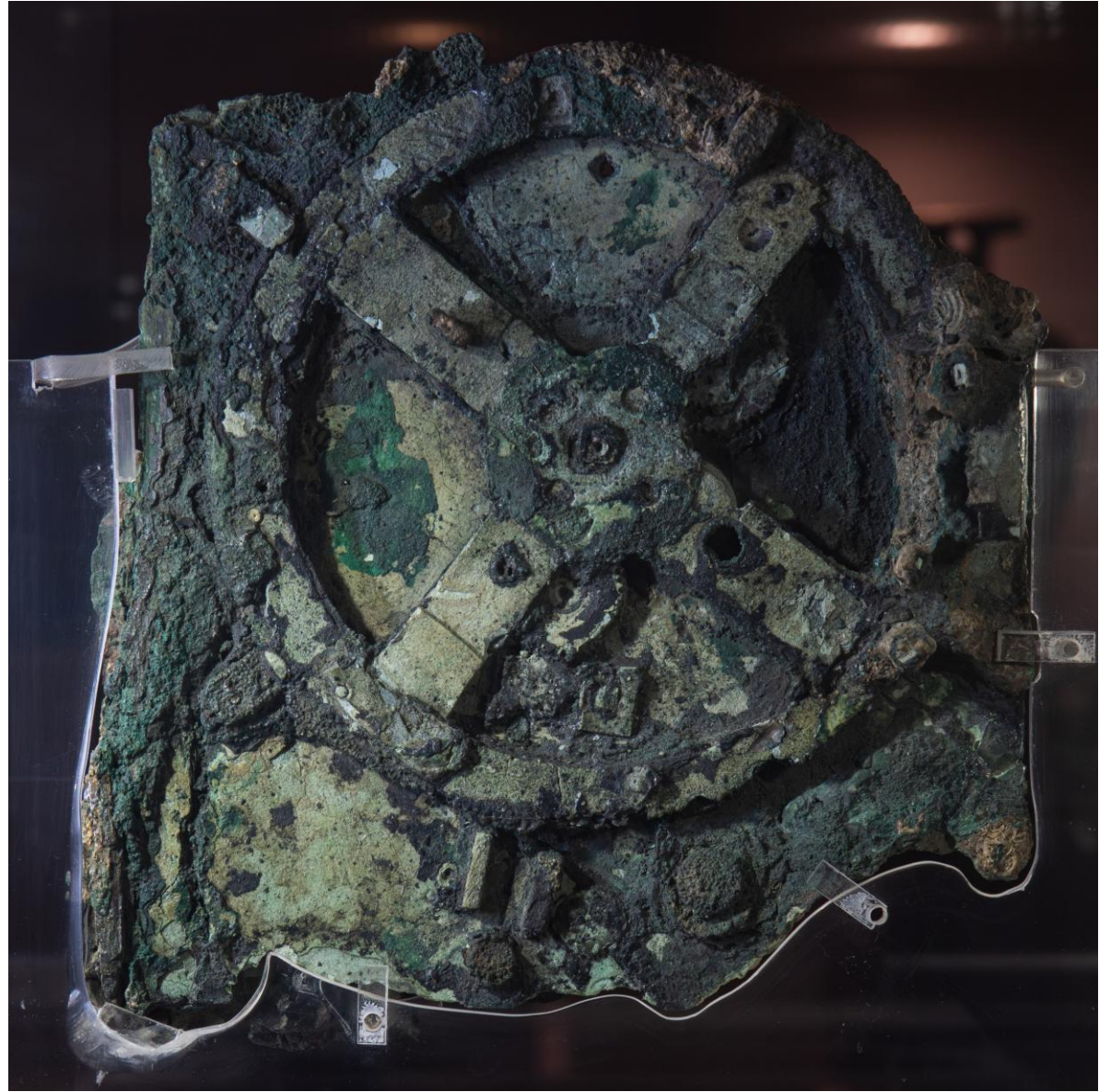
# Computer Architecture: What *is* a computer?

Dr. Colin O'Flynn

# Solving Problems in History



~2100 years ago







# Mechanical, Analog, etc

- These computers are built to do a specific task
- Sometimes can be reconfigured by changing gears etc, but not a general-purpose computer.

# Turing Complete Machines



A system is **Turing complete** if it can compute any algorithm, given enough time and memory.

# e.g., NOT Turing complete

- A “straight line” machine capable of calculations *without* branches:

**x = a + b;**

**y = x \* 3;**

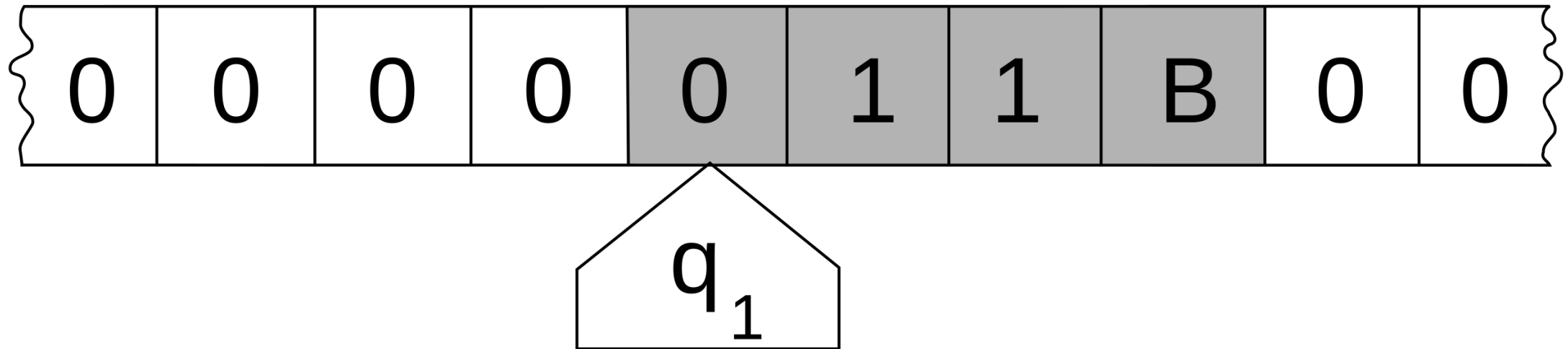
**z = y - 1;**

# Minimum Viable Turing Machine

- State (something that changes)
- Conditional behavior
- Unbounded repetition
- Ability to read/write memory



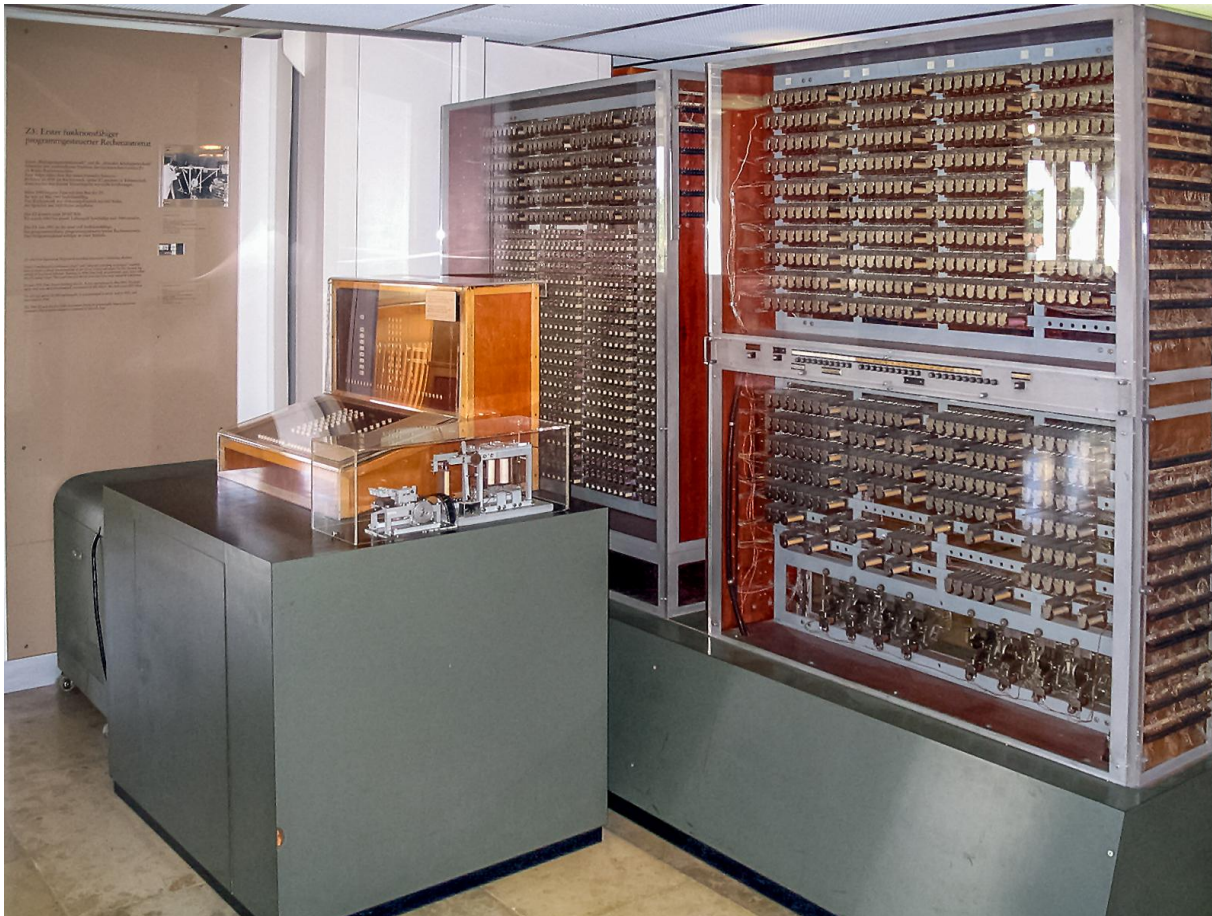
# Minimum Viable Turing Machine



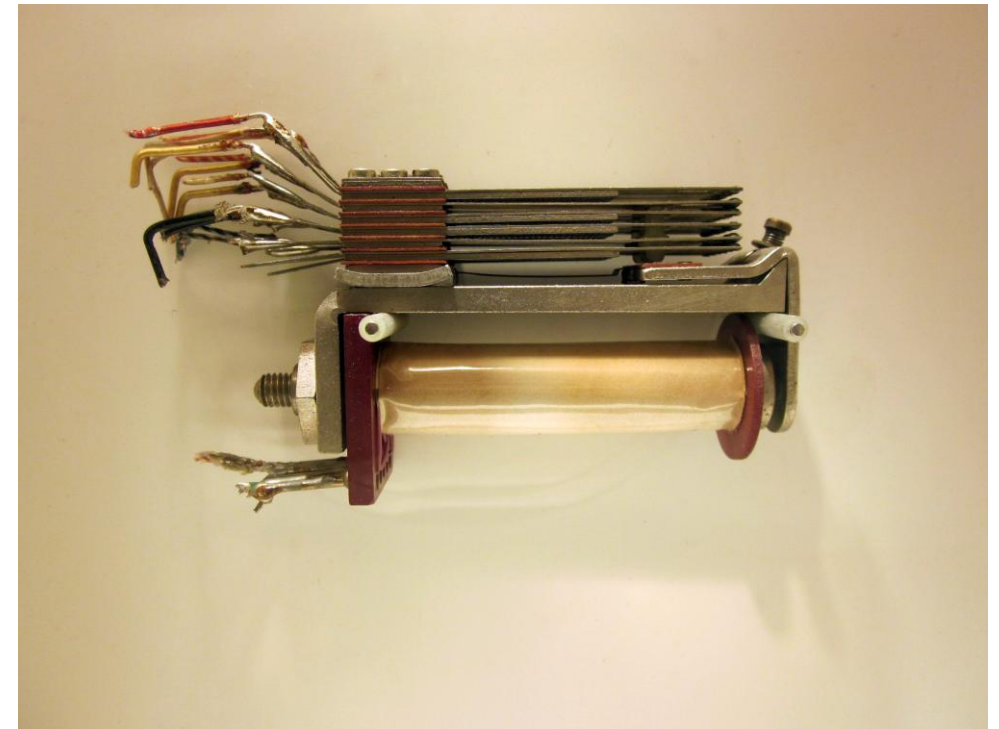
# e.g., Turing Complete Things

- C/C++
- Arm Assembly Code
- PowerPoint
- Excel
- The x86 “mov” instruction

# 1st Turing Complete Machine (ish)



The Zuse Z3 (1941)

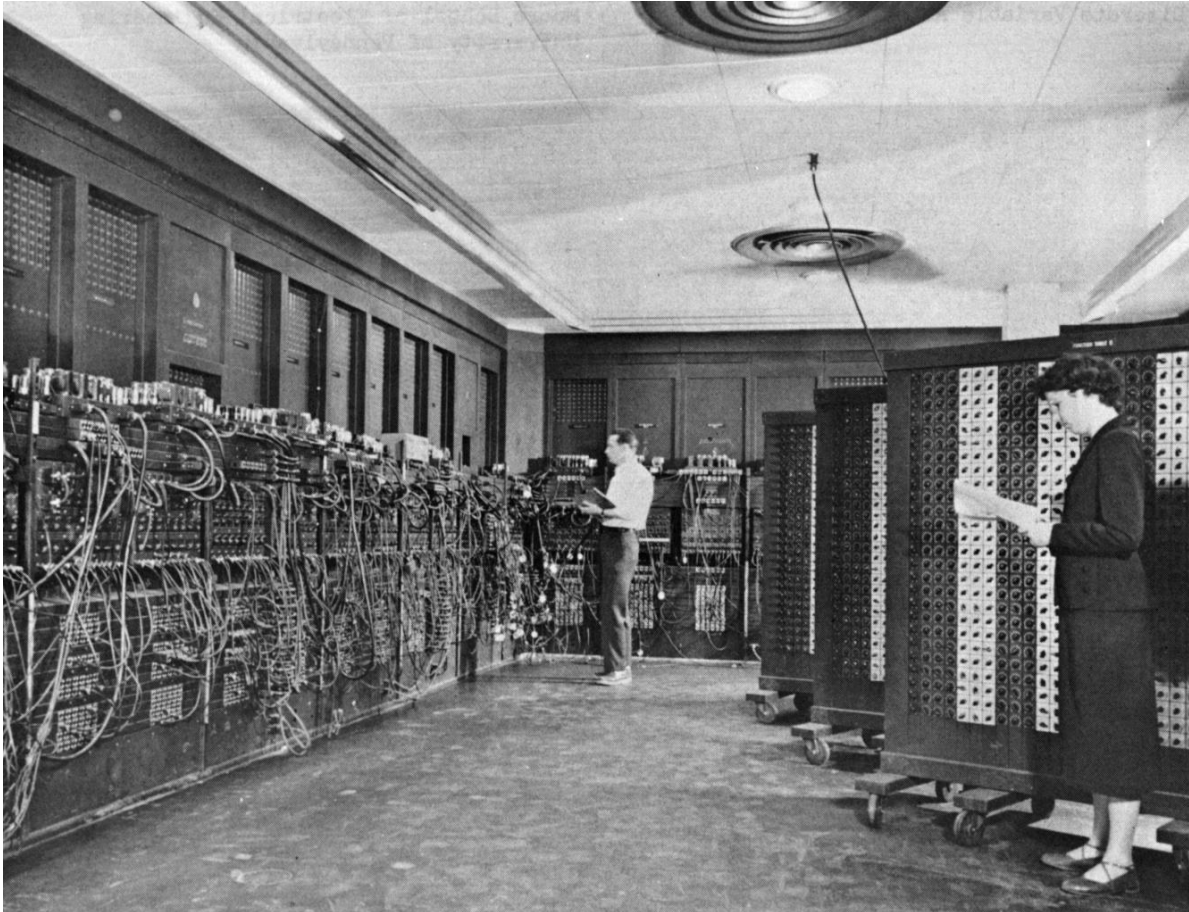


# Zuse Z3

- ISA Summary:
  - Two registers (R1, R2)
  - Load memory to register
  - Arithmetic operation on R1 & R2
  - Store R1 & R2 to memory
- No conditional branch instruction
  - The machine isn't *by design* Turing-complete – as a bit of a hack it can be *made* Turing-complete by running operations to cancel out some.
- Despite this – shows early idea of an *instruction set*



# ENIAC



- Designed for artillery firing tables
- Online in December 1945
- Included a **conditional branch** – this was the first Turing-complete machine *by design* in operation.

# Ultimate RISC

- It is possible to build a computer with one instruction.
- Called One Instruction Set Computer (OISC)
  - There is *many of them*: [https://en.wikipedia.org/wiki/One-instruction\\_set\\_computer](https://en.wikipedia.org/wiki/One-instruction_set_computer)
- E.g., FlipJump

# FlipJump

<https://esolangs.org/wiki/FlipJump>

*Opcode: Flips a bit, then jump (unconditionally)*

Arguments: a;b

***not \*a; jump b***

# e.g., a FlipJump program

1000;256 // addresses 000-127

32;446 // addresses 128-255

128;256 // addresses 256-383

(Open Excel Example Program)

→ Because flipping a bit can change jump, we ultimately get *storage, conditional execution, etc*

# C to FlipJump Compiler

- <https://github.com/tomhea/c2fj>
- Yes you can compile C code to this!



# x86 “mov” instruction

<https://github.com/LeetArxiv/ImplementedPapers/blob/main/mov-is-turing-complete.pdf>

## Comparison of $R_i$ & $R_j$ :

```
mov  $[R_i]$ , 0  
mov  $[R_j]$ , 1  
mov  $R_k$ ,  $[R_i]$ 
```

# x86 “mov” instruction as Comparison

- Remember – x86 is CISC, so *memory-register architecture*
- If  $R_i = R_j$  (e.g., == 123)

|                                                 |                                                                       |
|-------------------------------------------------|-----------------------------------------------------------------------|
| <code>mov [R<sub>i</sub>], 0</code>             | Store “ <b>0</b> ” to address 123                                     |
| <code>mov [R<sub>j</sub>], 1</code>             | Store “ <b>1</b> ” to address 123 (overwrite the “0”)                 |
| <code>mov R<sub>k</sub>, [R<sub>i</sub>]</code> | Load address 123 (which now has “ <b>1</b> ”) to <b>R<sub>k</sub></b> |

# x86 “mov” instruction as Comparison

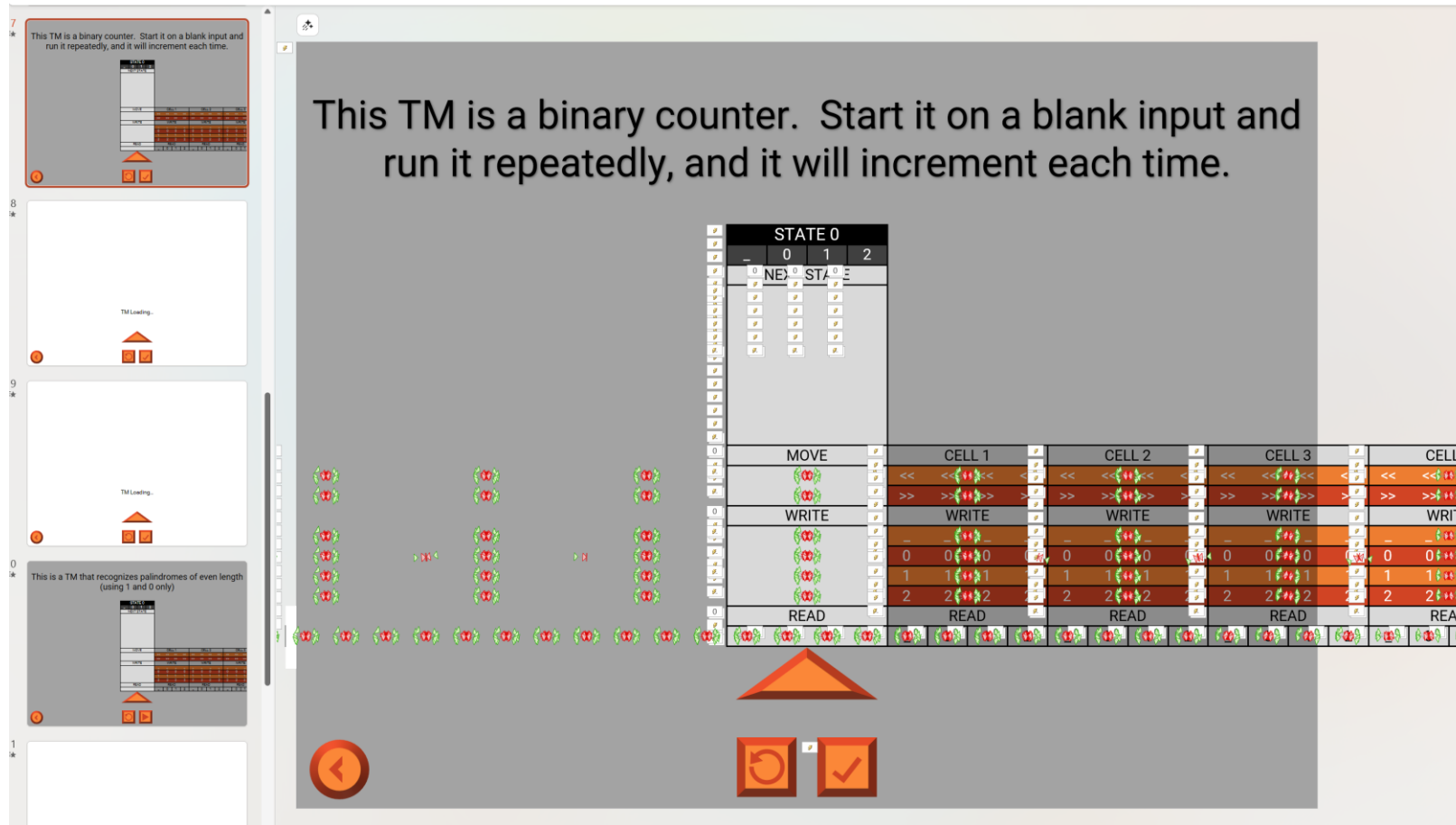
- Remember – x86 is CISC, so *memory-register architecture*
- If  $R_i \neq R_j$  (e.g.,  $R_i == 123$ ,  $R_j == 124$ )

|                                                 |                                                                   |
|-------------------------------------------------|-------------------------------------------------------------------|
| <code>mov [R<sub>i</sub>], 0</code>             | Store “ <b>0</b> ” to address 123                                 |
| <code>mov [R<sub>j</sub>], 1</code>             | Store “ <b>1</b> ” to address 124                                 |
| <code>mov R<sub>k</sub>, [R<sub>i</sub>]</code> | Load address 123 (which has “ <b>0</b> ”) to <b>R<sub>k</sub></b> |

# Turing Completeness?

- *Having a conditional normally means we can end up with a Turing complete machine.*

# PowerPoint as Turing Machine



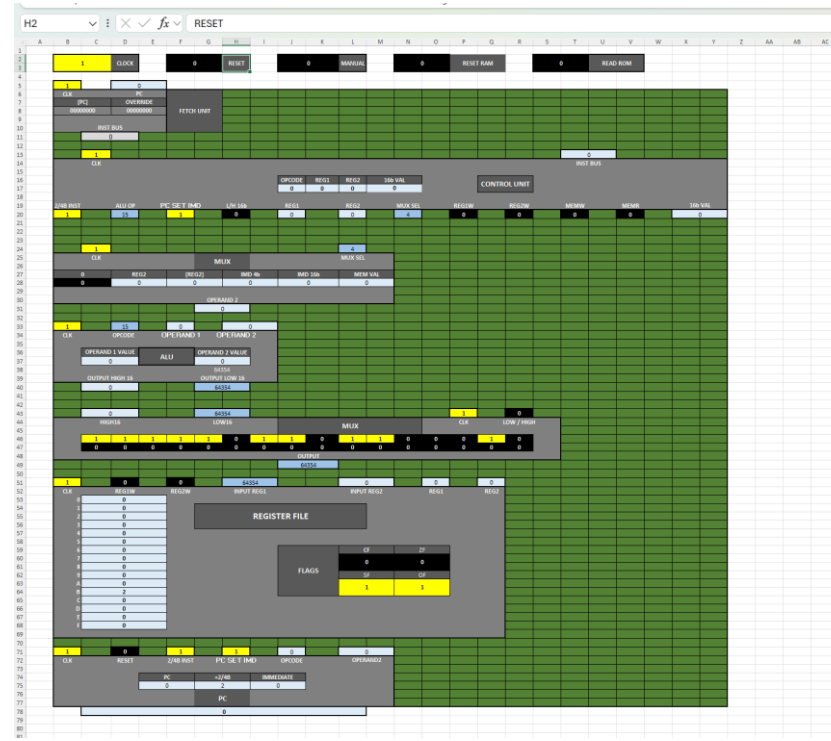
<https://www.youtube.com/watch?v=uNjxe8ShM-8> [https://esolangs.org/wiki/\(P%E2%84%A2TM%E2%84%A2\)%E2%84%A2](https://esolangs.org/wiki/(P%E2%84%A2TM%E2%84%A2)%E2%84%A2)

# Excel 16-bit Computer

<https://github.com/InkboxSoftware/excelCPU>

Yes you could be taught this course  
with *only excel*.

No this wouldn't be useful for your  
career!



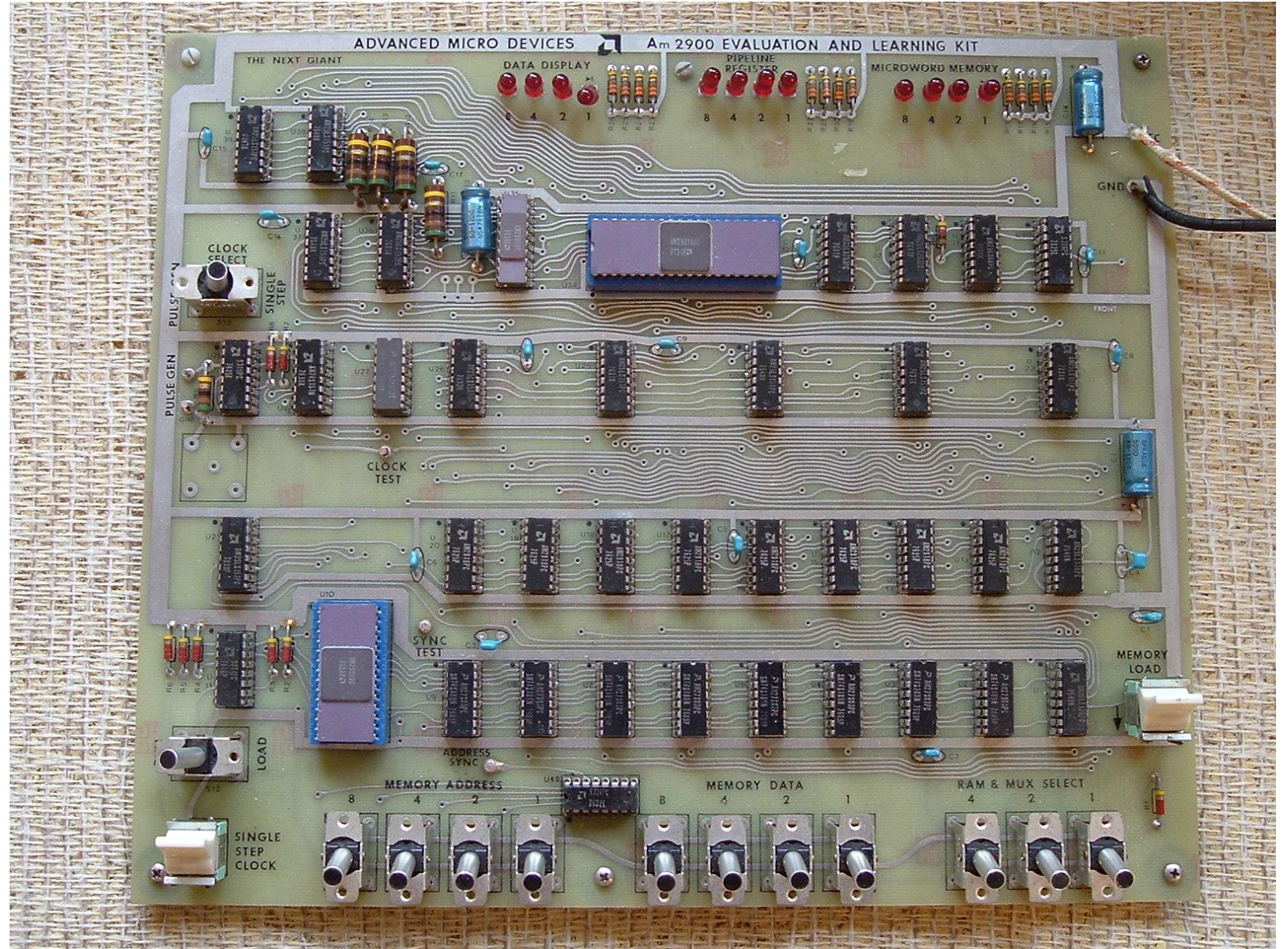


# Performance?

- None of this talks about performance.
- Most work you do in CA is related to making reasonable (useful) performance.
- (Almost) anything can compute – but how fast and with how many instructions is where most of our work is.

e.g.:  
Early CPU  
Trainers

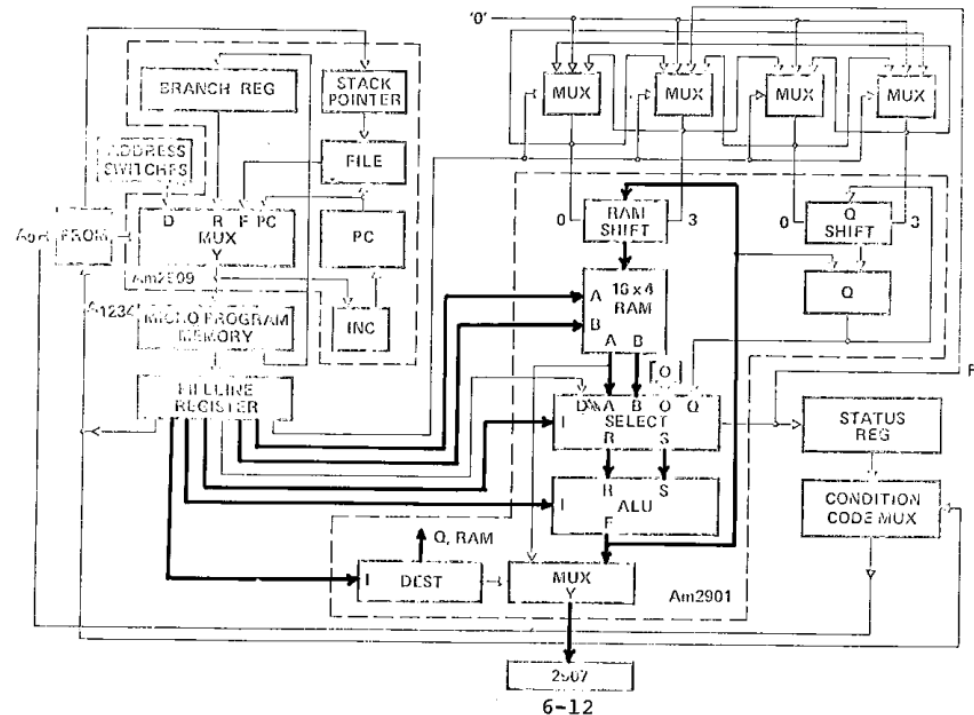
[https://www.donnamaie.com/AMD\\_Vintage/AMD\\_2900\\_ED2900A.html](https://www.donnamaie.com/AMD_Vintage/AMD_2900_ED2900A.html)



| RAM & MUX      | 7              | 6              | 5           | 4             | 3    | 2    | 1    | 0    | NOTES                |                                                  |
|----------------|----------------|----------------|-------------|---------------|------|------|------|------|----------------------|--------------------------------------------------|
|                |                |                |             |               |      |      |      |      | TEXT ADDRESS CONTROL | DATA CONTROL                                     |
| MEMORY ADDRESS | FRANCH ADDRESS | INSTR. ADDRESS | TEST SELECT | SOURCE SELECT | ALU  | 'A'  | 'B'  | 'O'  |                      |                                                  |
| 0              |                |                | 011         | 011           | 1000 |      | 0000 |      |                      | INC R <sub>0</sub>                               |
| 1              |                |                | 011         | 011           | 0001 |      | 0000 |      |                      | DEC R <sub>0</sub>                               |
| 2              |                |                | 011         | 101           | 0000 | 0000 | 0000 | 0011 |                      | R <sub>0</sub> + 3                               |
| 3              |                |                |             |               |      |      |      |      |                      |                                                  |
| 4              |                |                |             |               |      |      |      |      |                      |                                                  |
| 5              |                |                |             |               |      |      |      |      |                      |                                                  |
| 6              |                |                |             |               |      |      |      |      |                      |                                                  |
| 7              |                |                | 011         | 011           | 100  |      | 0000 |      |                      | CLR R <sub>0</sub>                               |
| 8              |                |                | 011         | 111           | 011  |      | 0001 | 0101 |                      | R <sub>1</sub> = 5                               |
| 9              |                |                | 011         | 001           | 0000 | 0001 | 0000 |      |                      | R <sub>0</sub> ← R <sub>0</sub> + R <sub>1</sub> |
| 10             |                |                |             |               |      |      |      |      |                      |                                                  |
| 11             |                |                |             |               |      |      |      |      |                      |                                                  |
| 12             |                |                |             |               |      |      |      |      |                      |                                                  |
| 13             |                |                |             |               |      |      |      |      |                      |                                                  |
| 14             |                |                |             |               |      |      |      |      |                      |                                                  |
| 15             |                |                |             |               |      |      |      |      |                      |                                                  |

BLANK - DON'T CARE

INSTRUCTION  $R_0 \leftarrow R_0 + R_1$



| Binary<br>Code | P <sub>3</sub> P <sub>2</sub> P <sub>1</sub> P <sub>0</sub> | Function                            |
|----------------|-------------------------------------------------------------|-------------------------------------|
| 0              | 0 0 0 0                                                     | Branch Register if F≠0              |
| 1              | 0 0 0 1                                                     | Branch Register                     |
| 2              | 0 0 1 0                                                     | Continue                            |
| 3              | 0 0 1 1                                                     | Branch Map (D Switches)             |
| 4              | 0 1 0 0                                                     | Jump-to-Subroutine if F≠0           |
| 5              | 0 1 0 1                                                     | Jump-to-Subroutine                  |
| 6              | 0 1 1 0                                                     | Return-from-Subroutine              |
| 7              | 0 1 1 1                                                     | File Reference                      |
| 8              | 1 0 0 0                                                     | End Loop & Pop if F=0               |
| 9              | 1 0 0 1                                                     | Push (and continue)                 |
| 10             | 1 0 1 0                                                     | Pop (and continue)                  |
| 11             | 1 0 1 1                                                     | End Loop & Pop if C <sub>n+4</sub>  |
| 12             | 1 1 0 0                                                     | Branch Register if F=0              |
| 13             | 1 1 0 1                                                     | Branch Register if F <sub>3</sub>   |
| 14             | 1 1 1 0                                                     | Branch Register if OVR              |
| 15             | 1 1 1 1                                                     | Branch Register if C <sub>n+4</sub> |

FIGURE 3: Instructions in the Am29751 Next Address Control PROM Supplied in the Am2900 Kit



# Summary

- Turing Complete is the minimum viable requirement for something to execute arbitrary code.
- Turing Complete *does not* mean this is practical or speedy.
- Even early processors considered performance – the fundamentals we will cover have been considered since the *introduction of the field*